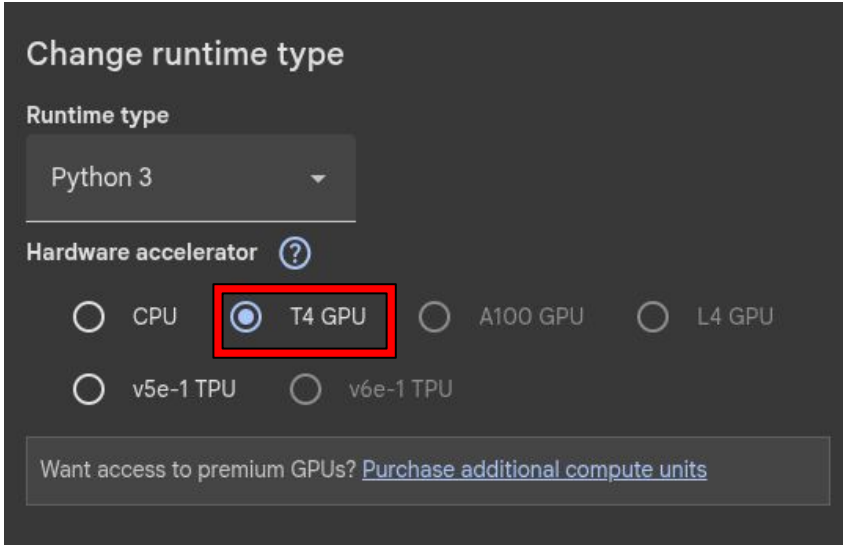
SA-MIRI 2025

Practice Pf: Deep Learning Basics (using Colab)

Jakub Seliga (jakub.seliga@estudiantat.upc.edu)
Thomas Aubertier (thomas.aubertier@estudiantat.upc.edu)

Task 7.1 Set up your google colab environment

- https://colab.research.google.com/
- Setup GPU usage
- Open/load notebook from GitHub
- Display execution time : [1] ✓ 8s



Task 7.2 Execute the provided notebook step-by-step

- “x_” variables are containing the raw data (each image), “y_” variables are containing the expected result.
- Two types of datasets : **train** (used to train the model using the expected result in back propagation) and **test** (new set used to compute a benchmark of the trained model).
- Model creation : add different layers

```
model = Sequential()
model.add(Input(shape=(28, 28, 1)))  # Define the input shape here
model.add(Conv2D(32, (5, 5), activation='relu'))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (5, 5), activation='relu'))
model.add(MaxPooling2D((2, 2)))
```

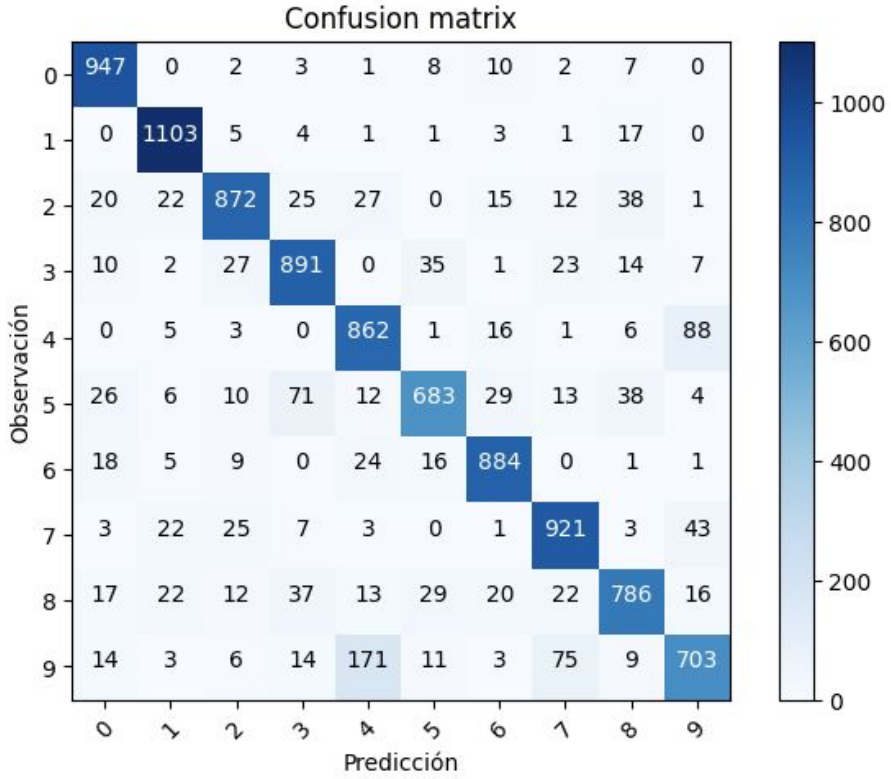
```
test_loss, test_acc = model.evaluate(x_test, y_test)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8476 - loss: 0.6420
```

- Accuracy : percentage of correct predictions on test dataset.

Task 7.2 Execute the provided notebook step-by-step

- Confusion matrix : gives the repartition of the error.
- 9s are more likely to be detected as 4s (false positive on 4, false negative on 9) than the opposite (false positive on 9, false negative on 4).
- Ideal : all data fall on the diagonal.



Task 7.3 – Improve the accuracy of your model

- For improving the model, we changed consecutively:
- increased the number of neurons in middle layer (10 ->
- added more dense layers and dropout, replaced sigmoid with tanh, replaced SGD with the adam optimizer, normalized the data using

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 10)	7,850
dense_1 (Dense)	(None, 10)	110

Total params: 7,960 (31.09 KB)
Trainable params: 7,960 (31.09 KB)
Non-trainable params: 0 (0.00 B)

Model: "sequential_2"

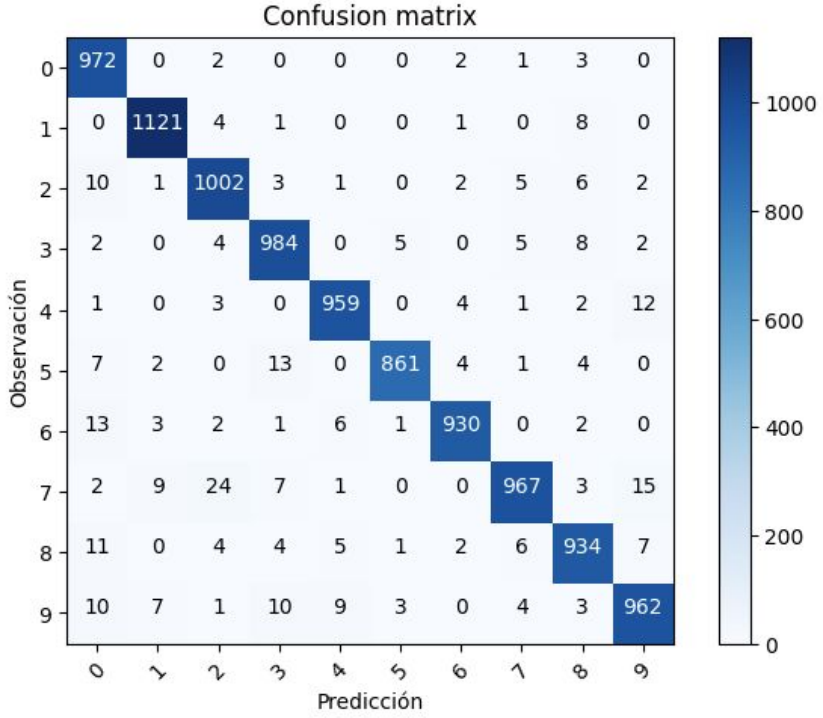
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 24, 24, 32)	832
max_pooling2d (MaxPooling2D)	(None, 12, 12, 32)	0
conv2d_1 (Conv2D)	(None, 8, 8, 64)	51,264
max_pooling2d_1 (MaxPooling2D)	(None, 4, 4, 64)	0
flatten (Flatten)	(None, 1024)	0
dense_2 (Dense)	(None, 10)	10,250

Total params: 62,346 (243.54 KB)
Trainable params: 62,346 (243.54 KB)
Non-trainable params: 0 (0.00 B)

Task 7.3 – Improve the accuracy of your model

Initial : shape = 32/64

activation = “relu”



Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 24, 24, 32)	832
max_pooling2d (MaxPooling2D)	(None, 12, 12, 32)	0
conv2d_1 (Conv2D)	(None, 8, 8, 64)	51,264
max_pooling2d_1 (MaxPooling2D)	(None, 4, 4, 64)	0
flatten (Flatten)	(None, 1024)	0
dense_2 (Dense)	(None, 10)	10,250

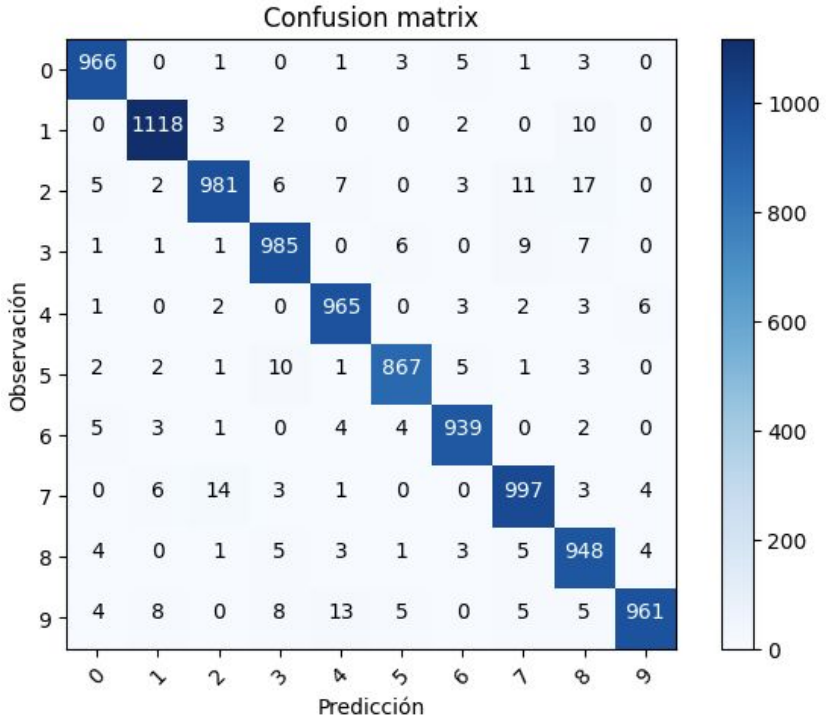
Total params: 62,346 (243.54 KB)
Trainable params: 62,346 (243.54 KB)
Non-trainable params: 0 (0.00 B)

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9643 - loss: 0.1231
Test accuracy: 0.9692000150680542
```

Task 7.3 – Improve the accuracy of your model

Bigger : shape = **64**/**128**

activation = “relu”



Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 24, 24, 64)	1,664
max_pooling2d_2 (MaxPooling2D)	(None, 12, 12, 64)	0
conv2d_3 (Conv2D)	(None, 8, 8, 128)	204,928
max_pooling2d_3 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten_1 (Flatten)	(None, 2048)	0
dense_1 (Dense)	(None, 10)	20,490

Total params: 227,082 (887.04 KB)
Trainable params: 227,082 (887.04 KB)
Non-trainable params: 0 (0.00 B)

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9698 - loss: 0.1112
Test accuracy: 0.9726999998092651
```

Task 7.3 – Improve the accuracy of your model

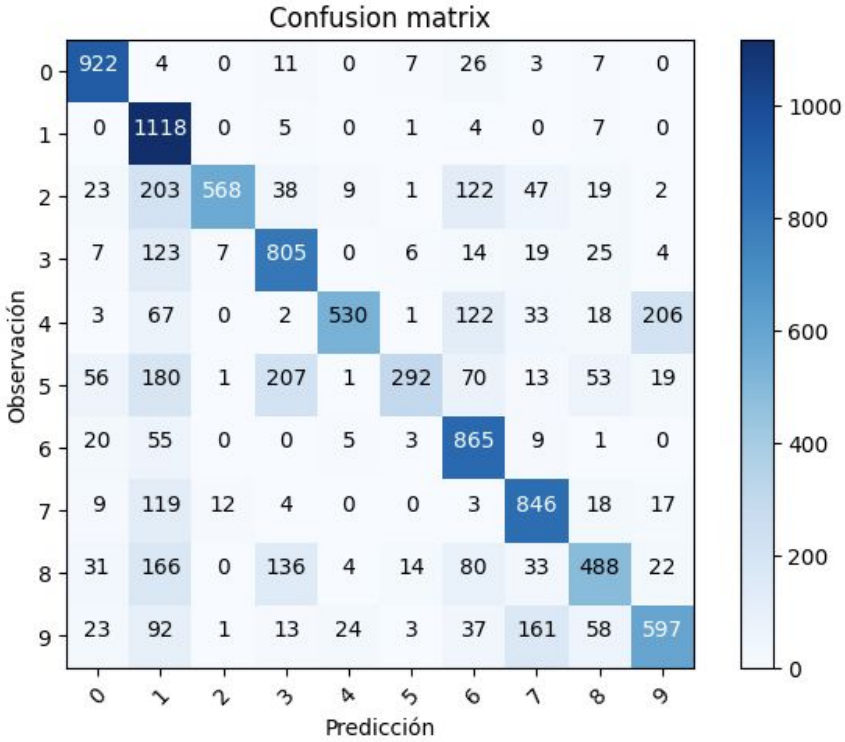
Different function : shape = 32/64

activation = “**sigmoid**”

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 24, 24, 32)	832
max_pooling2d_6 (MaxPooling2D)	(None, 12, 12, 32)	0
conv2d_7 (Conv2D)	(None, 8, 8, 64)	51,264
max_pooling2d_7 (MaxPooling2D)	(None, 4, 4, 64)	0
flatten_3 (Flatten)	(None, 1024)	0
dense_3 (Dense)	(None, 10)	10,250

Total params: 62,346 (243.54 KB)
Trainable params: 62,346 (243.54 KB)
Non-trainable params: 0 (0.00 B)




```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.6753 - loss: 1.6318
Test accuracy: 0.7031000256538391
```

Task 8.1 Improving a Basic CNN Model

- Starting with following configuration :

➔ `Conv2D` (size = 32, activation = “relu”)

➔ `MaxPooling2D`

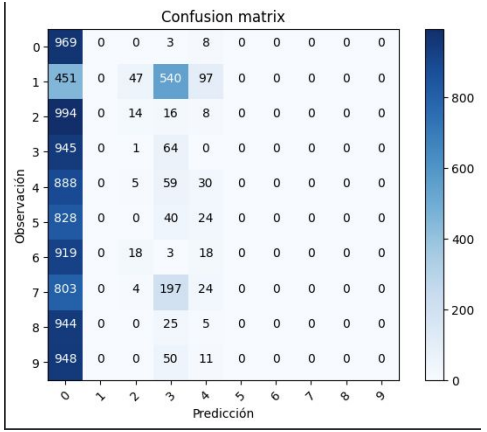
➔ `Conv2D` (size = 64, activation = “relu”)

➔ `MaxPooling2D`

➔ `Dense` (activation = “softmax”)

➔ Optimizer : `sgd`

Task 8.1 Improving a Basic CNN Model



- `softmax`, with this construction, is essential (avoid concentration of 0s/9s).
- Bigger size -> better results, more time.
- `relu` top performances by ≈ 1% (`linear`, `tanh` very close)
- Optimizers (with same model) : `sgd` 97.0%, `adam` 99.1%, `adagrad` 91.6%, `rmsprop` 99.1%, `adadelta` 62.9%, `nadam` 98.9% -> `adam`, `rmsprop` and `nadam`, `sgd` close.
- Second fit does improve the accuracy on `train` dataset, but not necessarily on `test` dataset (same cases are failing)

```
Epoch 1/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.8624 - loss: 0.4795
Epoch 2/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9806 - loss: 0.0633
Epoch 3/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9873 - loss: 0.0418
Epoch 4/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9899 - loss: 0.0313
Epoch 5/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9925 - loss: 0.0252
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9880 - loss: 0.0398
Test accuracy: 0.9904000163078308
```

```
Epoch 1/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9931 - loss: 0.0223
Epoch 2/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9950 - loss: 0.0169
Epoch 3/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9961 - loss: 0.0118
Epoch 4/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9962 - loss: 0.0118
Epoch 5/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9971 - loss: 0.0085
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9893 - loss: 0.0366
Test accuracy: 0.991100013256073
```

Task 8.2 Exporting the python script

- Setup used in .py script :

➔ `Conv2D` (size = 64, activation = “relu”)

➔ `MaxPooling2D`

➔ `Conv2D` (size = 128, activation = “relu”)

➔ `MaxPooling2D`

➔ `Dense` (activation = “softmax”)

➔ Optimizer : `adam`

`cnn_train.py`

Task 8.3 Running your first natural network on a login node

- Note : trainings on MN5 were done using a downloaded copy of `mnist.npz`.

```
from tensorflow.keras.utils import to_categorical

def load_data(path):
    with np.load(path) as f:
        x_train, y_train = f['x_train'], f['y_train']
        x_test, y_test = f['x_test'], f['y_test']
        return (x_train, y_train), (x_test, y_test)

mnist = tf.keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = load_data(path='mnist.npz')
```

Task 8.3 Running your first natural network on a login node

- Overall accuracy : 98.9% ≈ 99.1% => expected result
- Note : using size 64/128, process is killed.

```
Epoch 1/5
600/600 [==============================] - 3s 5ms/step - loss: 0.2079 - accuracy: 0.9407
Epoch 2/5
600/600 [==============================] - 3s 5ms/step - loss: 0.0591 - accuracy: 0.9816
Epoch 3/5
600/600 [==============================] - 3s 4ms/step - loss: 0.0411 - accuracy: 0.9879
Epoch 4/5
600/600 [==============================] - 3s 5ms/step - loss: 0.0320 - accuracy: 0.9897
Epoch 5/5
600/600 [==============================] - 3s 5ms/step - loss: 0.0259 - accuracy: 0.9920
313/313 [==============================] - 1s 2ms/step - loss: 0.0355 - accuracy: 0.9893
Test accuracy: 0.9893000125885010
```

Task 8.4 Submitting your first gpu dl training with slurm

- Overall accuracy : 98.9%
  => expected result

```
[nct01029@alogin1 ~]$ cat cnn_train_31652188.out
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 24, 24, 64)        1664

 max_pooling2d (MaxPooling2D  (None, 12, 12, 64)       0
 )

 conv2d_1 (Conv2D)           (None, 8, 8, 128)         204928

 max_pooling2d_1 (MaxPooling  (None, 4, 4, 128)        0
 2D)

 flatten (Flatten)           (None, 2048)              0

 dense (Dense)               (None, 10)                20490

=================================================================
Total params: 227,082
Trainable params: 227,082
Non-trainable params: 0
_________________________________________________________________
(60000, 28, 28)
(60000,)
(60000, 28, 28, 1)
(60000, 10)
Epoch 1/5
600/600 [==============================] - 5s 7ms/step - loss: 0.1618 - accuracy: 0.9538
Epoch 2/5
600/600 [==============================] - 4s 7ms/step - loss: 0.0463 - accuracy: 0.9856
Epoch 3/5
600/600 [==============================] - 4s 7ms/step - loss: 0.0319 - accuracy: 0.9900
Epoch 4/5
600/600 [==============================] - 4s 7ms/step - loss: 0.0234 - accuracy: 0.9927
Epoch 5/5
600/600 [==============================] - 4s 7ms/step - loss: 0.0185 - accuracy: 0.9941
313/313 [==============================] - 1s 2ms/step - loss: 0.0330 - accuracy: 0.9893
Test accuracy: 0.989300012588501
```

```
#!/bin/bash
#SBATCH --job-name=cnn_train
#SBATCH -o %x_%J.out
#SBATCH -e %x_%J.err

#SBATCH --nodes=1
#SBATCH --time=00:15:00
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=20
#SBATCH --gres=gpu:1
#SBATCH --exclusive

#SBATCH --account=nct_345
#SBATCH --qos=acc_debug

module purge
module load singularity

SINGULARITY_CONTAINER=/gpfs/projects/nct_345/MN5-NGC-TensorFlow-23.03.sif

singularity exec $SINGULARITY_CONTAINER python cnn_train.py
```

Task 9.1 Comparative Implementation in PyTorch and TensorFlow

- PyTorch took significantly more time to load.

TensorFlow:

```
[5]
✓ 2s
(x_trainTF_, y_trainTF_), _ = tf.keras.datasets.mnist.load_data()

x_trainTF = x_trainTF_.reshape(60000, 784).astype('float32')/255
y_trainTF = tf.keras.utils.to_categorical(y_trainTF_, num_classes=10)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
```

PyTorch:

```
[6]
✓ 9s
xy_trainPT = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=torchvision.transforms.Compose([torchvision.transforms.ToTensor()]))

xy_trainPT_loader = torch.utils.data.DataLoader(xy_trainPT, batch_size=batch_size)
```

```
100%|██████████| 9.91M/9.91M [00:02<00:00, 4.93MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 131kB/s]
100%|██████████| 1.65M/1.65M [00:01<00:00, 1.24MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 11.7MB/s]
```

Task 9.1 Comparative Implementation in PyTorch and TensorFlow

- TensorFlow's Dense layer declare activation+block in one line unlike PyTorch.
- PyTorch's size is constructed as (input_size, output_size).
- Loss/optimizer works identically (not shown here).

TensorFlow:

```
modelTF = tf.keras.Sequential([
                            tf.keras.layers.Dense(10,activation='sigmoid',input_shape=(784,))
                            tf.keras.layers.Dense(10,activation='softmax')
          ])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

PyTorch:

```
modelPT=torch.nn.Sequential(torch.nn.Linear(784,10),
                            torch.nn.Sigmoid(),
                            torch.nn.Linear(10,10),
                            torch.nn.LogSoftmax(dim=1)
                           )
```

Task 9.1 Comparative Implementation in PyTorch and TensorFlow

- `TensorFlow` training is more compact, is more automated.
- `PyTorch` lasted thrice as long.

TensorFlow:

[13] ✓ 22 s

```
_ = modelTF.fit(x_trainTF, y_trainTF, epochs=epochs, batch_size=batch_size, verbose = 0)
```

PyTorch:

[14] ✓ 1 min

```
for e in range(epochs):
    for images, labels in xy_trainPT_loader:
        images = images.view(images.shape[0], -1)
        loss = criterion(modelPT(images), labels)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
```

Task 9.1 Comparative Implementation in PyTorch and TensorFlow

- TensorFlow testing is more compact, is more automated. In particular, PyTorch accuracy is computed “by hand”.
- Results are very close : for this example at least, TensorFlow did perform a bit better and took 4x less time.

TensorFlow:

[15] ✓ 1 s

```
_, (x_testTF, y_testTF)= tf.keras.datasets.mnist.load_data()
x_testTF = x_testTF.reshape(10000, 784).astype('float32')/255
y_testTF = tf.keras.utils.to_categorical(y_testTF, num_classes=10)

_ , test_accTF = modelTF.evaluate(x_testTF, y_testTF, verbose=0)
print('\nTensorFlow model Accuracy =', test_accTF)
```

```
TensorFlow model Accuracy = 0.8762000203132629
```

PyTorch:

[16] ✓ 4 s

```
xy_testPT = torchvision.datasets.MNIST(root='./data', train=False,
xy_test_loaderPT  = torch.utils.data.DataLoader(xy_testPT)

correct_count, all_count = 0, 0
for images,labels in xy_test_loaderPT:
  for i in range(len(labels)):
    img = images[i].view(1, 784)

    logps = modelPT(img)
    ps = torch.exp(logps)
    probab = list(ps.detach().numpy()[0])
    pred_label = probab.index(max(probab))
    true_label = labels.numpy()[i]
    if(true_label == pred_label):
      correct_count += 1
    all_count += 1

print("\nPyTorch model Accuracy  =", (correct_count/all_count))
```

```
PyTorch model Accuracy  = 0.8615
```